



Docker

para desarrolladores ...

...

by M.T.I. Juan Luis Campos Barrera

Temas

- ¿Qué es Docker y qué es un Contenedor?
- Docker vs. Máquina Virtual
- Instalación de Docker (Versión Desarrollador)
- Comandos Básicos
- Volúmenes (Persistencia de datos)
- Dockerfile (Construyendo nuestras imágenes propias)
- Docker Compose (Corriendo múltiples servicios)
-

¿Que es Docker?

¿Qué es Docker y qué problemas resuelve?



Repositorio de
Contenedores



Desarrollo de
Aplicaciones



Deploy en
aplicaciones

¿Qué es Docker?



Representa una forma para **empaquetar** aplicaciones, con TODAS las dependencias necesarias y su configuración



Artefacto Portable que sea fácil de compartir y mover



Lograr un desarrollo y deploy de aplicaciones más eficientes

¿Qué es Docker?

¿Dónde se almacenan?



Categories

- Networking
- Security
- Languages & frameworks
- Integration & delivery
- Message queues
- API management
- Internet of things
- Machine learning & AI
- Developer tools
- Data science
- Web servers
- Operating systems
- Content management system
- Databases & storage
- Monitoring & observability
- Web analytics

Machine Learning & AI

IMAGE + 1 MORE



tensorflow/tensorflow
tensorflow

Official Docker images for the machine learning framework TensorFlow (<http://www.tensorflow.org>)

🕒 16h ⬇️ 50M+ ☆ 2861

IMAGE



pytorch/pytorch
pytorch

PyTorch is a deep learning framework that puts Python first.

🕒 2m ⬇️ 10M+ ☆ 1604

IMAGE



langchain/langchain
LangChain

⚡ Building applications with LLMs through composability ⚡

🕒 2y ⬇️ 50K+ ☆ 362

IMAGE + 1 MORE



python
Docker Official Image

Python is an interpreted, interactive, object-oriented, open-source programming language.

🕒 5d ⬇️ 1B+ ☆ 10387

Trending this week

MODEL



ai/qwen3
Docker

Qwen3 is the latest Qwen LLM, built for top-tier coding, math, reasoning, and language tasks.

🕒 4m ⬇️ 500K+ ☆ 121

IMAGE



arm32v7/redis
arm32v7

Redis is the world's fastest data platform for caching, vector search, and NoSQL databases.

🕒 13d ⬇️ 5M+ ☆ 34

IMAGE



hylang
Docker Official Image

Hy is a Lisp dialect that translates expressions into Python's abstract syntax tree.

🕒 5d ⬇️ 10M+ ☆ 64

IMAGE



atlassian/confluence
Atlassian

🕒 2d ⬇️ 10M+ ☆ 95

Most pulled images

[View all](#)

¿En que ayudan a mejorar el desarrollo?

...

veamos como era antes de los Contenedores ...



Desarrollador



Desarrollador



antes de los Contenedores ...



Cada sistema operativo tiene un proceso distinto para la instalación, incluso librerías



Son muchos pasos que hay que seguir para instalar todo lo necesario

⚠ Se corre el riesgo de hacer mal la instalación



Esto puede llegar a ser tedioso, tardado ... aburrido, etc



ahora con los Contenedores . . .



Con su propio ambiente aislado



Se empaqueta con toda la configuración necesaria



Se utiliza la versión exacta para cada dependencia



Con un solo comando se crea todo



¿En qué ayudan a mejorar el deploy?

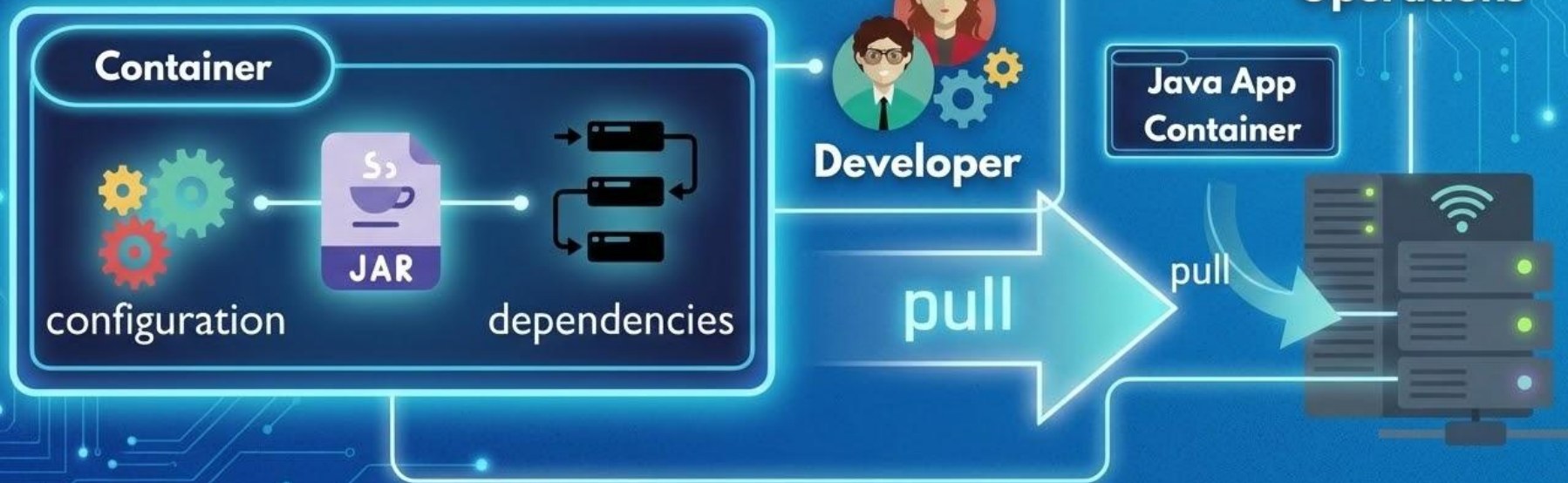
...

despliegue de aplicaciones en producción

antes de los contenedores ...



deploy con Contenedores



¿Qué es un Contenedor?

Concepto técnico



Docker vs Virtual Machine

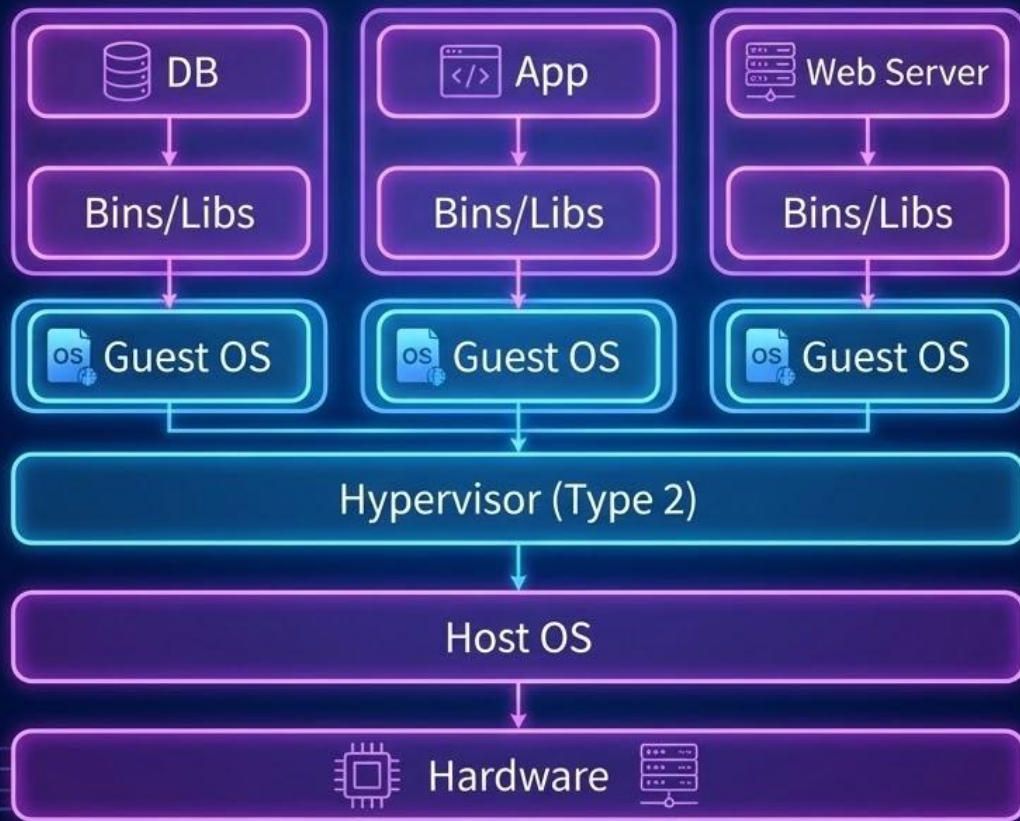
Virtualización

Existen bastantes herramientas para realizar virtualización, por ejemplo:

- VirtualBox
- VMware vSphere
- Xen
- Hyper-V
- KVM

y en todas, el proceso es el mismo: Crear Partición, Asignar Memoria RAM, Instalar Sistema Operativo, instalar servicios y dependencias, para finalmente montar nuestra aplicación.

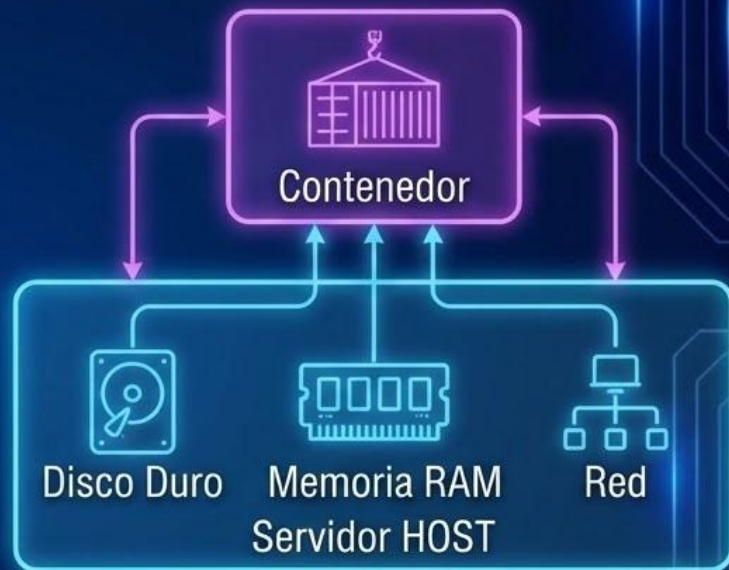
Arquitectura de Máquina Virtual



Contenedor

Proveedores Populares:

- Linux Containers
- LXC
- LXD
- **Docker**
- GCManager (deprecated)
- Portainer
- Containerd
- Windows Server Containers



Un contenedor, utiliza los recursos de servidor HOST, Disco Duro, Memoria RAM, Red, etc.

Máquinas Virtuales vs. Contenedores

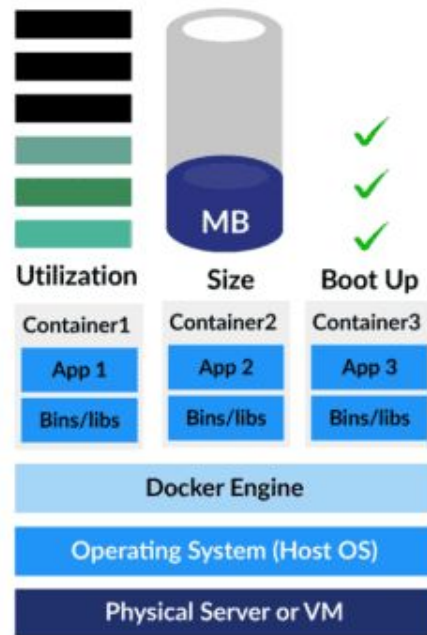
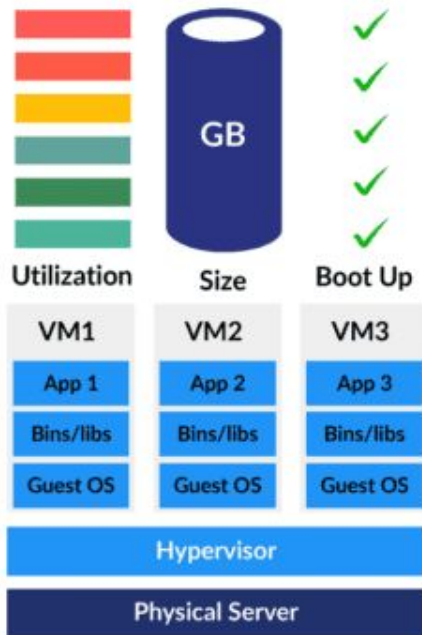
Máquinas Virtuales



Contenedores



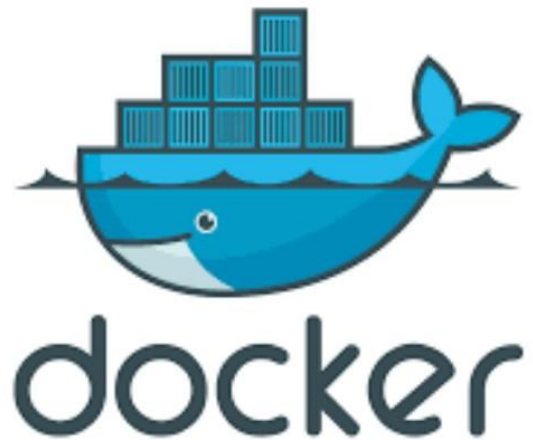
en resumen



Instalación

versión desarrollador...

<https://docs.docker.com/get-docker/>





Docker

Comandos Básicos ...

Subtemas

Conceptos Clave



Container vs Image



Versión y Tag

Comandos Docker

```
> docker pull [image]
> docker run [options] [image]
> docker start [container]
> docker stop [container]
> docker ps [-a]
> docker exec -it [container] [command]
> docker logs [container]
```

Comandos Imagen y containers



Contenedor

- Es un entorno de ejecución para una Imagen
 - ⚙️ se requiere **configurar** su puerto, su red, su persistencia, etc.
 - 🔗 **conectar** con otros contenedores

Listar Contenedores que están corriendo:

```
> docker container ls = docker ps
```

Listar Todos los contenedores

```
> docker ps -a
```

Imagen



Servicio o Aplicación lista para ser utilizada por un Contenedor

Listar Imagenes

```
> docker images
```

Descargar una imagen

```
> docker pull [imagen_name][:versión]
```

Comandos



Correr Imagen Determinada

```
# Sintaxis
docker run imagen[:versión]

# Ejemplo
docker run nginx:alpine
```



Correr en Background (Detach)

```
# Sintaxis
docker run -d
imagen[:versión]
# Ejemplo
docker run -d nginx:alpine
```



Correr con Puerto Específico

```
# Sintaxis
docker run -p
host_port:container_port image

# Ejemplo
docker run -d -p 8080:80
nginx:alpine
```

Comandos



Detener un contenedor

```
docker stop ID_Container
```



Iniciar

```
docker start ID_Container
```



Eliminar | Forzar eliminación

```
docker rm ID_Container |  
docker rm -f ID_Container
```

Comandos



Ver Logs

```
docker logs ID_Container
```



Logs en tiempo real

```
docker logs -f ID_Container
```



Nombrar contenedor

```
docker run -d -p 8080:80 --name  
web_test1 nginx:1.18
```

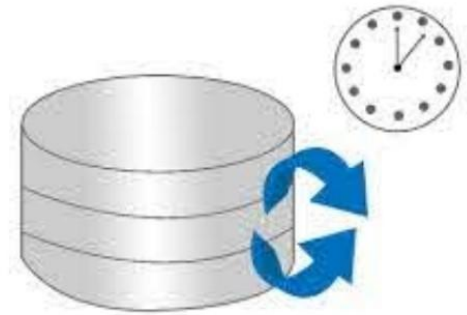


Entrar en un contenedor

```
docker exec -it ID_Container  
[bash|sh]
```

Volumenes

Persistencia de Datos



Volúmenes



Es la manera sencilla y predefinida para almacenar todos los datos de un contenedor y asegurarnos de que persistan hasta que decidamos eliminarlos.

Puntos de acceso que va desde un punto en concreto de nuestro host, hasta un punto en concreto dentro de nuestro contenedor

```
docker run -d --name web2 -p 8080:80 -v $PWD:/usr/share/nginx/html nginx
```

Build Images

Construir nuestras propias
Imágenes



Demo app en django

Primera Opción

Dentro de un contenedor de ubuntu, instalar lo necesario para correr una aplicación de django, un simple Hola Mundo.



Dentro de un contenedor de ubuntu, instalar lo necesario para correr una aplicación de django, un simple Hola Mundo.



```
docker run -it ubuntu bash
```

actualizar repositorios
instalar dependencias



Dockerfile

Segunda Opción:



El archivo *Dockerfile* (exactamente como se escribe), es prácticamente una receta de cocina, contiene las instrucciones precisas que se debe seguir para construir una imagen de Docker.

- 🚢 a partir de este archivo construimos nuestra imagen
- 🚢 el archivo es mucho más fácil de compartir
- 🚢 Tiene una estructura definida.

Dockerfile

Estructura:

-  **FROM:** capa origen, ejemplo alpine, ubuntu, python....etc
-  **RUN:** ejecuta órdenes dentro del contenedor, dependiendo del origen que hemos elegido son las ordenes, ejemplo: para ubuntu apt install , y para alpine apk add, etc
-  **ENV:** Agregar variables de entorno
-  **COPY:** Copia archivos desde nuestro HOST al contenedor
-  **ADD:** Copia y además descomprime archivos dentro del contenedor
-  **WORKDIR:** Cambia a un directorio dentro del contenedor
-  **EXPOSE:** Expone un puerto en particular ejemplo 80, 3000, 8080, 8000
-  **CMD:** Ejecuta scripts o comandos para correr los servicios o aplicaciones

Docker Compose

Corriendo múltiples servicios



Docker Compose

Compose es una herramienta para definir y ejecutar aplicaciones Docker de varios contenedores. Con Compose, utiliza un archivo tipo YAML para configurar los servicios de su aplicación. Ayudando a configurar, volúmenes, puertos, dependencias entre servicios, etc. Luego, con un solo comando, crea e inicia todos los servicios desde su configuración.

Antes que nada ... Como se instala

Debian, Ubuntu, Mintetc.



```
sudo apt-get install docker-compose
```

Fedora:



```
sudo dnf -y install docker-compose
```

Mac/Windows



Se instala junto con Docker  

Estructura archivo docker-compose.yml



version:

actualmente existen 3 versión, con sus respectivas variantes, afecta la versión en la sintaxis y la cantidad de parámetros que podemos utilizar



services:

dentro de este parámetro se declaran los servicios



volumes: se declaran los volúmenes a utilizar dentro del compose y que deben estar declarados en los servicios



networks:

establecemos si los servicios pertenecen a una determinada red

Estructura archivo docker-compose.yml



image:

se declara el nombre de la imagen a utilizar para el servicio, ya sea local y de algún registry al que tengamos acceso



container_name:

este parámetro es opcional, pero siempre es bueno poder identificar de mejor manera el nombre de nuestro contenedor, de no utilizarlo se genera un nombre aleatorio



depends_on:

indica la dependencia con otro servicio, de existir, espera a que se genere primero el servicio dependiente



links:

nos ayuda a referenciar de una forma más amigable a un servicio

Estructura archivo docker-compose.yml



build: indica que este servicio se debe construir dentro del proyecto

context: indica la ubicación donde está el Dockerfile

dockerfile: nombre del archivo



volumes: lo podemos usar para indicarle que archivos y/o carpetas del proyecto estarán ligadas al servicio



environment & env_file:

environment: dentro se declaran todas las variables de entorno necesarias

env_file: se pueden declarar las variables en un archivo externo

Estructura archivo docker-compose.yml



restart: políticas de reinicio del servicio, las opciones son:

- **“no”** : que no se reinicie
- **always:** siempre se reinicia
- **on-failure:** cuando existe una falla
- **unless-stopped:** similar a always, pero no se reinicia si lo detenemos

Estructura archivo docker-compose.yml



expose:

Expone un puerto del contenedor de nuestro servicio



ports:

Relaciona un puerto de nuestra máquina host con un contenedor, depende de la disponibilidad del puerto en el host



entrypoint:

ejecuta un script en cuanto el servicio está listo

Comandos docker-compose



up: levanta los servicios y muestra los logs



ps: muestra los contenedores corriendo y sus estados



up -d: levanta los servicios en background (detach)



down: termina todos los servicios (con -v elimina volúmenes)



up -d --build: construye imágenes y luego levanta en background



logs -f: muestra logs en tiempo real



build: solo construye las imágenes declaradas



restart: reinicia todos los servicios, o uno específico